CAS Correctness and Progress Analysis

EPAM COE Research Notes

1. What is CAS?

CAS stands for **Compare-And-Swap**. It is an atomic operation used in concurrent programming. CAS checks whether a shared value equals an expected value. If it does, CAS replaces it with a new value; otherwise, it fails.

Concept: CAS(variable, expected, newValue) → if variable == expected, update it; otherwise, fail.

2. Why is CAS Needed?

When multiple threads update the same variable, a race condition can produce an incorrect result. If two threads both read counter = 0 and then write 1, the final result becomes 1 instead of 2. CAS makes the update conditional and atomic.

Without protection: Read 0 → Read 0 → Write 1 → Write 1 → Final = 1 (expected 2).

3. How CAS Works

Suppose counter = 10. Thread 1 wants to change it to 11 using CAS(10, 11). If the counter is still 10, CAS succeeds. If another thread has changed it to 12, CAS fails because the expected value no longer matches.

Success: expected = 10, actual = 10 → value becomes 11.
Failure: expected = 10, actual = 12 → no update.

4. CAS Retry Loop

CAS is often used inside a loop: read the current value, calculate a new value, try CAS, and retry if another thread changed the value first.

Read value → Calculate new value → CAS → Success: finish | Failure: read again and retry.

5. CAS Correctness

The comparison and update happen as one atomic operation. Only one competing thread can successfully change a value from a particular expected state. Other threads detect the change and can retry using the latest value.

The successful CAS is often the **linearization point**—the moment when the operation logically takes effect.

6. ABA Problem

A thread reads A, another thread changes A → B → A, and the first thread sees A again. It may assume nothing changed, although the value changed in between. Some algorithms use version numbers or stamped references to detect this.

7. What is Progress?

Correctness is not enough; an algorithm should also keep making progress when threads compete. If Thread 1's CAS fails but Thread 2 succeeds, Thread 1 did not progress, but the system as a whole did. This is the basic idea of **lock-free progress**.

8. Lock-Free vs Wait-Free

A **lock-free** algorithm guarantees system-wide progress, even if an individual thread repeatedly fails. A **wait-free** algorithm gives a stronger guarantee: every thread completes its operation within a bounded number of steps.

Property	Meaning
Lock-Free	System continues making progress; one thread may repeatedly retry.
Wait-Free	Every thread completes within a bounded number of steps.

9. CAS Performance and Contention

CAS is non-blocking, but it is not always faster. If many threads update the same variable, many CAS attempts may fail. Repeated retries consume CPU time and reduce performance. Benchmarks should be used to compare implementations.

10. Simple Experiment

Create several threads, each incrementing a shared counter many times. Compare a normal counter, a synchronized counter, and an AtomicInteger/CAS-based counter.

Implementation	Correctness	Typical behavior
Normal counter	May fail	Race condition possible
Synchronized/lock	Correct	Threads may block
Atomic/CAS	Correct if properly used	Non-blocking; retries possible

11. Key Findings

• CAS performs an atomic compare-and-update.
• CAS failure usually means another thread changed the value.
• Retry loops let an algorithm try again with the latest value.
• Successful CAS can be the linearization point.
• The ABA problem matters in some algorithms.
• Lock-free is system-wide progress; wait-free is stronger.
• High contention can cause repeated failures and lower performance.

12. Conclusion

CAS is a fundamental building block for lock-free concurrent programming. It helps maintain correctness without traditional locks, but good designs must also consider progress and performance. CAS provides the foundation; careful algorithm and data-structure design makes a system reliable and efficient.